

Amirkabir University of Technology
(Tehran Polytechnic)

Amirkabir University of Technology
Department of Mathematics and Computer Science

Deep Learning

Name:

Student Number:

Due Date:

Assignment: Number 2

Problem 1 Assume that we have data points in the interval $[0, 1] \in \mathbb{R}$ belonging to three possible classes. We are looking for possible soft classifiers.

- a) Recall why such classifiers are functions $p : \mathbb{R} \rightarrow \mathbb{R}^3$.
- b) Show that the following two functions are acceptable classifiers:

$$p(x) = \left(\frac{x}{2}, \frac{x^2}{2}, 1 - \frac{x}{2} - \frac{x^2}{2}\right),$$
$$\tilde{p}(x) = \left(1 - 4x(1 - x) - x(1 - 2x)^2, 4x(1 - x), x(1 - 2x)^2\right).$$

- c) Is the function

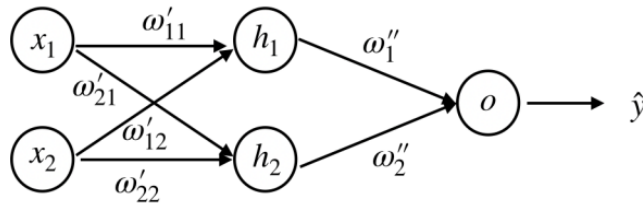
$$q(x) = (x, 1 - x, (x - 1/2)^2)$$

an acceptable classifier?

- d) Consider the following data points in $\mathbb{R} : x_1 = 0$ in class # 1, $x_2 = 1/2$ in class # 2, and $x_3 = 1$ in class # 3. Calculate the results of the two classifiers p and $\tilde{p}$ on those data points. Which of the two has the best performance?
-

Problem 2 Consider a binary classification problem for which we are given an input vector $\{\mathbf{x}_i\}_{i=1}^n$ and we want to predict an output variable $y_i \in \{0, +1\}$ for each input $\mathbf{x}_i$. We use a neural network such as the one below, with parameters $\mathbf{w}'$, $\mathbf{w}''$, a sigmoid activation function for both the hidden layer and output layer and a cross-entropy loss function for each data point $\mathbf{x}_i$ defined as

$$L(\mathbf{w}', \mathbf{w}'') = -y_i \log \hat{y}_i(\mathbf{x}_i; \mathbf{w}', \mathbf{w}'') - (1 - y_i) \log(1 - \hat{y}_i(\mathbf{x}_i; \mathbf{w}', \mathbf{w}'')),$$



where $\hat{y}^i(\mathbf{w}', \mathbf{w}'') \in [0, 1]$ is the output of the neural network.

Using the forward pass, derive the loss function for an input $\mathbf{x} = (0.2, 0.8)$ and weights $w'_{11} = w'_{22} = 0.1$, $w'_{12} = 0.2$, $w'_{21} = 0.3$, $w''_1 = w''_2 = 0.2$, and for the target label $y = 0$.

Problem 3

- a) How many parameters are there in a deep neural network with a single input, a single output, and K hidden layers, each containing D hidden units?
 - b) Consider two neural networks that map a scalar input x to a scalar output y . The first network is shallow and has $D = 95$ hidden units. The second is deep and has $K = 10$ layers, each containing $D = 5$ hidden units. How many parameters does each network have? How many linear regions can each network make? Which would run faster?
-

Problem 4 We only consider in this exercise ANNs with ReLU as activation function for all neurons.

- a) Construct a neural network with a 1D input layer, a 1D output layer, and exactly one hidden layer such that the network function is exactly

$$f : x \in \mathbb{R} \rightarrow f(x) = |x| \in \mathbb{R}.$$

What is the minimal dimension of the hidden layer?

- b) Extend the previously constructed network to obtain the network function

$$f : x \in \mathbb{R}^n \rightarrow f(x) = (|x_1|, |x_2|, \dots, |x_n|) \in \mathbb{R}^n.$$

- c) We now want to prove that it is not possible to obtain exactly the network function, even with an arbitrary number of hidden layers

$$f : x \in \mathbb{R} \rightarrow f(x) = x^2 \in \mathbb{R}.$$

- i) Assume that all biases in the network are 0. Prove that in this case, the network function f necessarily satisfies

$$f(\lambda x) = |\lambda| f(x), \forall \lambda \in \mathbb{R}$$

so that $f(x) \neq x^2$.

- ii) In the general case with biases, prove that a network function is necessarily piecewise affine.
 - d) Can any piecewise linear function on $\mathbb{R}$ be obtained as a network function?
-

Problem 5 We consider a set Ω of objects, i. e., points of $\mathbb{R}^n$, that are in two different classes or subsets Ω_1 and Ω_2 . We assume that Ω_1 and Ω_2 are separated by a hyperplane: there exist $\alpha \in \mathbb{R}^n$ and $\beta \in \mathbb{R}$ such that

$$x \in \Omega_1 \implies \alpha \cdot x + \beta < 0, \quad x \in \Omega_2 \implies \alpha \cdot x + \beta > 0.$$

- a) Construct an ANN with input of dimension n and output of dimension 2 such that the network function $f(x) = (f_1(x), f_2(x))$ classifies Ω_1 and Ω_2 in the following sense:

$$x \in \Omega_1 \implies f_1(x) > f_2(x), \quad x \in \Omega_2 \implies f_1(x) < f_2(x).$$

- b) Extend the network constructed above to the case of four classes Ω_i , $i = 1, \dots, 4$, separated by two non-parallel hyperplanes: there exist $\alpha, \tilde{\alpha} \in \mathbb{R}^n$, and $\beta, \tilde{\beta} \in \mathbb{R}$, such that α and $\tilde{\alpha}$ are not collinear and

$$\begin{aligned} x \in \Omega_1 &\implies \alpha \cdot x + \beta < 0 \text{ and } \tilde{\alpha} \cdot x + \tilde{\beta} < 0, \\ x \in \Omega_2 &\implies \alpha \cdot x + \beta > 0 \text{ and } \tilde{\alpha} \cdot x + \tilde{\beta} < 0, \\ x \in \Omega_3 &\implies \alpha \cdot x + \beta < 0 \text{ and } \tilde{\alpha} \cdot x + \tilde{\beta} > 0, \\ x \in \Omega_4 &\implies \alpha \cdot x + \beta > 0 \text{ and } \tilde{\alpha} \cdot x + \tilde{\beta} > 0. \end{aligned}$$

Problem 6

- a) Prove that the following property holds for $\alpha \in \mathbb{R}^+$:

$$\text{ReLU}[\alpha \cdot z] = \alpha \cdot \text{ReLU}[z].$$

This is known as the non-negative homogeneity property of the ReLU function.

- b) Using the non-negative homogeneity property of the ReLU function, show that:

$$\text{ReLU}[\beta_1 + \lambda_1 \cdot \Omega_1 \text{ReLU}[\beta_0 + \lambda_0 \cdot \Omega_0 x]] = \lambda_0 \lambda_1 \cdot \text{ReLU}\left[\frac{1}{\lambda_0 \lambda_1} \beta_1 + \Omega_1 \text{ReLU}\left[\frac{1}{\lambda_0} \beta_0 + \Omega_0 x\right]\right]$$

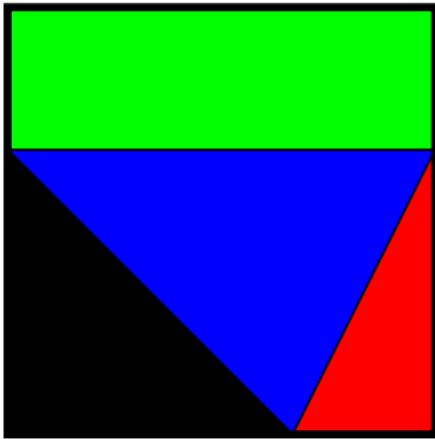
where λ_0 and λ_1 are non-negative scalars. From this, we see that the weight matrices can be rescaled by any magnitude as long as the biases are also adjusted, and the scale factors can be re-applied at the end of the network.

Problem 7

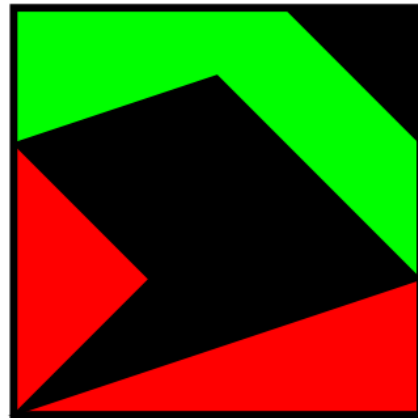
- a) How many neurons do you need in the output layer if you want to classify email into spam or ham? What activation function should you use in the output layer?
- b) If instead you want to tackle MNIST, how many neurons do you need in the output layer, and which activation function should you use?
- c) What about for getting your network to predict housing prices

- d) Provide an example where you could use the hyperbolic tangent ($\tanh$) activation function for the output layer.
 - e) Discuss the advantages and disadvantages of using the sigmoid activation function compared to the softmax activation function for binary classification tasks.
 - f) Explain the difference between using a linear activation function and a sigmoid activation function for regression tasks. When might you prefer one over the other?
 - g) Suppose you have a multi-label classification problem where each example can belong to multiple classes simultaneously (e.g., image tagging). What modifications would you make to the output layer and activation function compared to a typical multi-class classification problem?
-

Problem 8 In this problem we want to see how neural networks might learn certain function "exactly" and how adding more layers allows us solve more complicated problems.



(a) Partition into convex polytopes



(b) Partition into general polytopes

- a) Show that any classification problem where each class occupies a distinct region in space, given by a single convex polytope (cf. Figure a) can be solved exactly by a neural network with 2 hidden layers.
 - b) Show that any classification problem where each class occupies a distinct region in space, given by a finite set of (possibly non-convex) polytopes (cf. Figure b) can be solved exactly by a neural network with 3 hidden layers
-